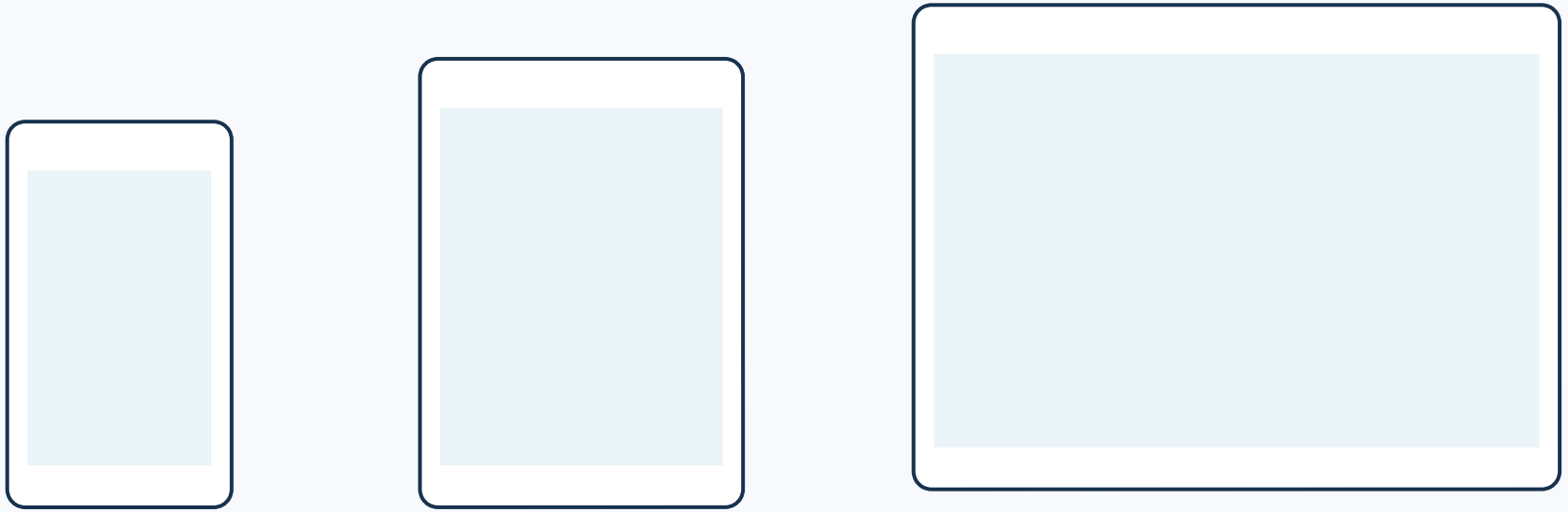
TD4 — Responsive design

Unités relatives • Media queries • Test sur différentes tailles d’écran



Même page HTML — présentation qui s’adapte

Déroulement de l’heure

Nous avançons ensemble, puis vous faites l’exercice en autonomie.

0–15 min	**A — Unités**	px, %, em, vw, calc()
15–35 min	**B — Media queries**	max-width, min-width, orientation
35–40 min	**Mode responsive**	simuler téléphone/tablette
40–57 min	**Exercice TD4**	adapter votre page du TD3
57–60 min	**Bilan**	vérification desktop + mobile

Étape 1 — Ouvrez les deux fichiers A

On commence par les unités. Ne modifiez rien pour l'instant.

Dans VS Code

```
A_units.html
A_units.css
```

Placez HTML et CSS côte à côte.
Puis ouvrez A_units.html dans le navigateur.

Ce que vous devez repérer

HTML

```
<div id="p50" class="child"></div>
```

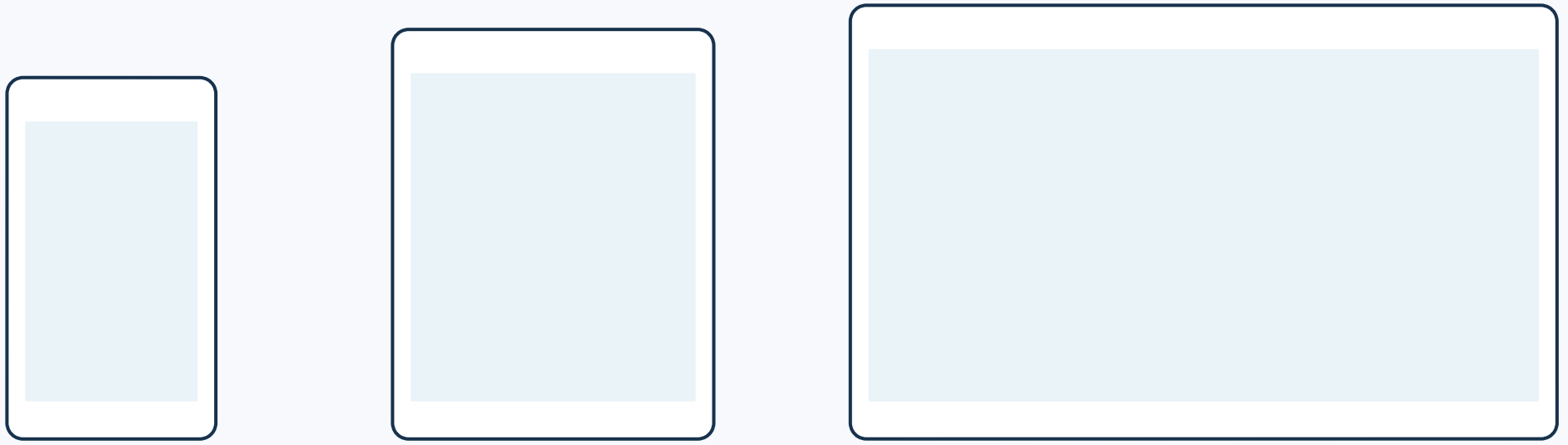
CSS

```
#p50 {
    height: 50%;
}
```

Même logique qu'avant : id="p50" → #p50

Avant de coder : que signifie « responsive » ?

Une page responsive adapte sa mise en page à l'espace disponible.



Téléphone

Tablette

Ordinateur / tactile

Le CSS ne demande pas : « est-ce un téléphone ? »

Il peut demander : « quelle est la largeur du viewport ? », « portrait ou paysage ? »

Test A1 — % : une taille relative au parent

Nous allons modifier UNE valeur puis observer.

HTML concerné

```
<div class="parent">
  ...
  <div id="p50"

class="child"></div>
</div>
```

Le bloc vert est un enfant de .parent.

CSS actuel

```
.parent {
  height: 200px;
}

#p50 {
  height: 50%;
}
```

Question à tester : que se passe-t-il si 50% devient 25% ?

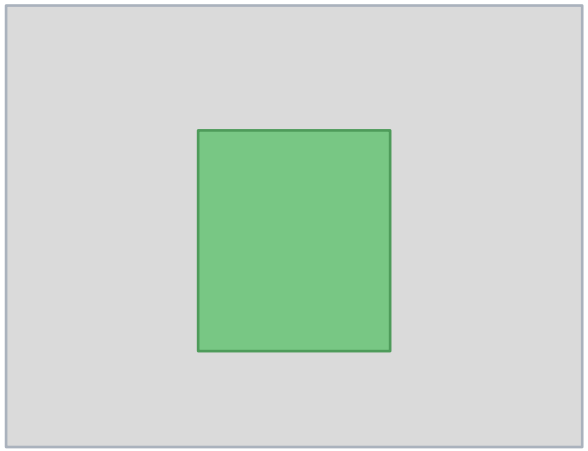
À FAIRE MAINTENANT

1. Dans A_units.css, trouvez #p50.
2. Remplacez 50% par 25%.
3. Ctrl+S puis actualisez.
4. Observez uniquement le bloc « height: 50% ».
5. Ne remettez pas encore 50%.

Observation A1 — le % dépend du parent

AVANT

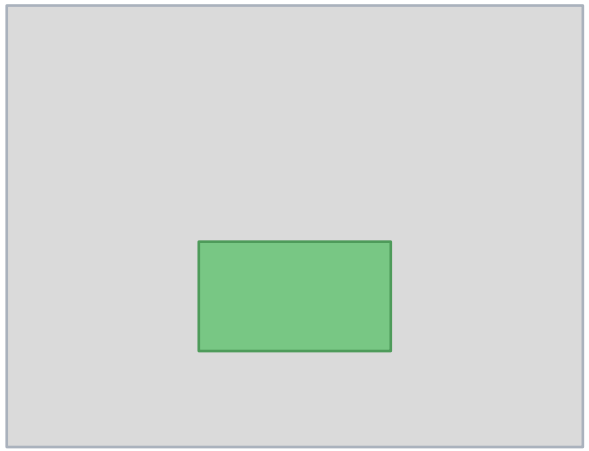
.parent = 200 px de haut



50% = 100 px

APRÈS

.parent = 200 px de haut



25% = 50 px

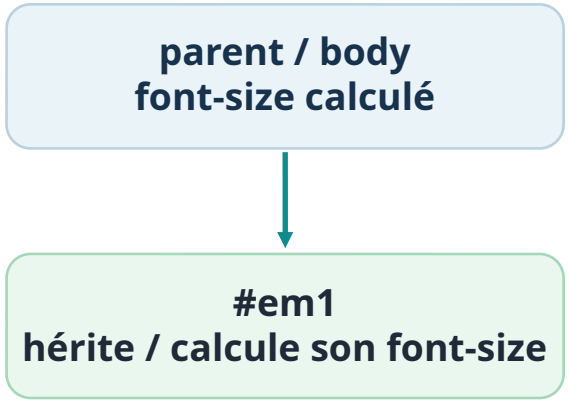
Maintenant : remettez #p50 à height: 50%;

Test A2 — em : la référence est la taille de police calculée

Pour height, margin, padding… 1em = 1 × font-size calculé de l'élément.

D'où vient le font-size ?

Si l'élément ne définit pas font-size, il l'hérite généralement de son parent.



Cas spécial : pour font-size: 2em, la référence est le font-size du parent.

```
#em1 {
  height: 1em;
}

#em2 {
  height: 2em;
}
```

Nous allons rendre la référence visible en ajoutant temporairement :

```
font-size: 30px;
```

À FAIRE MAINTENANT

1. Ajoutez font-size: 30px dans #em1.
2. Gardez height: 1em.
3. Ctrl+S puis actualisez.
4. Faites ensuite le même test sur #em2.
5. Comparez visuellement 1em et 2em.

Observation A2 — em suit le font-size de l’élément

#em1

```
font-size: 30px;
height: 1em;
```

1em = 30 px

#em2

```
font-size: 30px;
height: 2em;
```

2em = 60 px

% → référence : parent | em → référence : font-size calculé de l’élément

Remettez le fichier A_units.css comme au départ avant de continuer.

Test A3 — vw et calc()

Deux dernières idées : viewport et calcul de valeurs.

vw

```
#vw100 {
  width: 100vw;
}
```

1vw = 1% de la largeur du viewport.
100vw = toute la largeur visible.

calc()

```
#calc {
  height:
  calc(100% - 40px);
}
```

Le navigateur calcule : 100% de la référence moins 40px.

À FAIRE MAINTENANT

1. Sur #vw100 : remplacez 100vw par 50vw.
2. Réduisez aussi la largeur du navigateur.
3. Observez comment la largeur suit le viewport.
4. Sur #calc : remplacez - 40px par - 100px.
5. Observez puis remettez les valeurs d’origine.

Ce qu’il faut retenir des unités

Pas besoin de mémoriser toutes les unités CSS. Il faut savoir quelle est la référence.

`100px`	**valeur fixe**	ne dépend pas du parent
`50%`	**parent / bloc de référence**	change si le parent change
`2em`	**font-size calculé de l’élément**	change si la taille de police change
`50vw`	**largeur du viewport**	change quand la fenêtre change
`calc(...)`	**calcul entre valeurs**	combine plusieurs unités

Étape 2 — Ouvrez les deux fichiers B

On passe aux media queries.

Dans VS Code

```
B_media.html
B_media.css
```

Ouvrez B_media.html dans le navigateur.
Ne modifiez encore rien.

Important pendant les tests

Plusieurs boîtes peuvent devenir vertes en même temps.

C’est normal : plusieurs conditions peuvent être vraies simultanément.

À chaque test, je vous indique UNE boîte à observer. Ignorez les autres.

@media : une condition autour de CSS normal

```
@media (max-width: 768px) {

    #max768 {
        background-color: green;
    }

}
```

1 QUAND ? `@media (max-width: 768px)`

2 QUI ? `#max768`

3 QUOI ? `background-color: green;`

@media ne cible pas l'HTML. Le sélecteur à l'intérieur cible toujours l'HTML comme avant.

Test B1 — max-width

Pendant ce test, regardez uniquement la boîte #max768.

```
@media (max-width: 768px) {
    #max768 {
        background-color:
rgb(89,196,98);
    }
}
```

Dans le HTML :

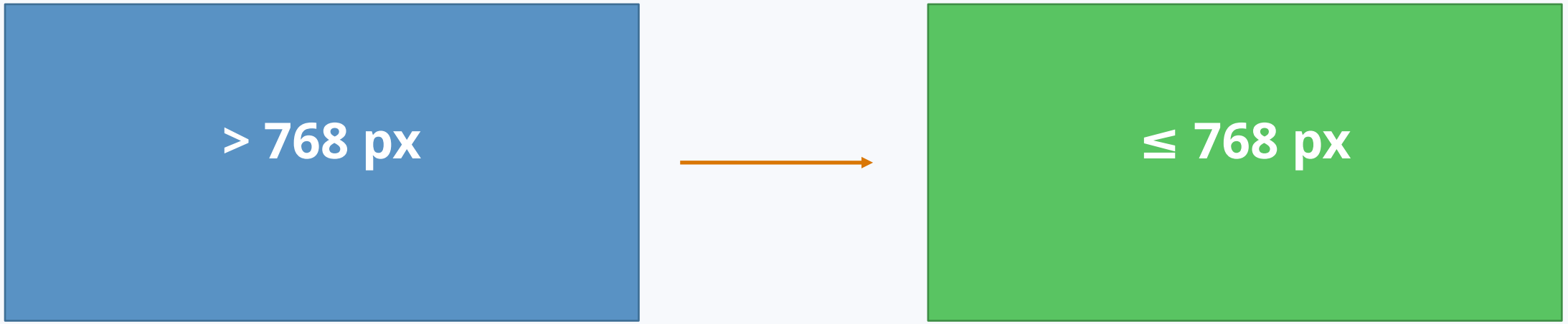
```
<div class="box" id="max768">...</div>
```

À FAIRE MAINTENANT

1. Placez le navigateur assez large.
2. Réduisez lentement sa largeur.
3. Regardez seulement #max768.
4. Repérez le moment où sa couleur change.
5. Ne modifiez pas encore le CSS.

Question pour la classe : à partir de quelle largeur la règle devient-elle active ?

Observation B1 — max-width fixe une limite supérieure



max-width: 768px → « largeur au maximum 768 px »

Mini-test : changez 768px → 500px, testez, puis remettez 768px.

Test B2 — min-width

Cette fois, regardez uniquement la boîte #min1024.

```
@media (min-width: 1024px) {
    #min1024 {
        background-color:
rgb(89,196,98);
    }
}
```

Dans le HTML :

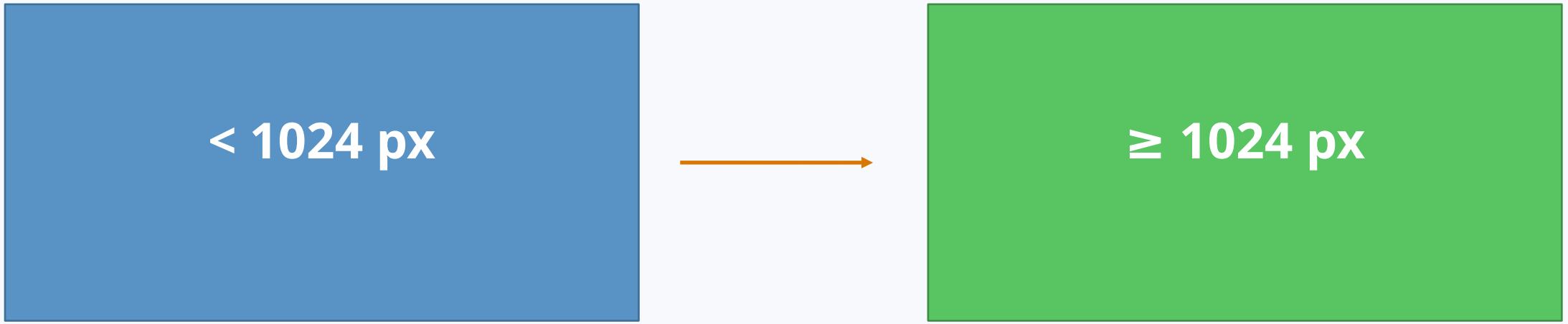
```
<div class="box" id="min1024">...</div>
```

À FAIRE MAINTENANT

1. Commencez avec une fenêtre étroite.
2. Agrandissez progressivement.
3. Regardez seulement #min1024.
4. Repérez le moment où sa couleur change.
5. Laissez le CSS inchangé.

Question : quel sens donne « min-width » à la condition ?

Observation B2 — min-width fixe une limite inférieure



min-width: 1024px → « largeur au minimum 1024 px »

Test B3 — une plage de largeurs

Regardez uniquement #min768max1024.

```
@media (min-width: 768px)
       and (max-width: 1024px) {
    #min768max1024 {
        background-color: green;
    }
}
```

Ne cherchez pas à mémoriser : lisez simplement les deux conditions.

À FAIRE MAINTENANT

1. Testez une largeur autour de 600px.
2. Puis autour de 900px.
3. Puis autour de 1200px.
4. Regardez uniquement #min768max1024.
5. Notez dans quel intervalle la boîte est verte.

Observation B3 — les deux conditions doivent être vraies

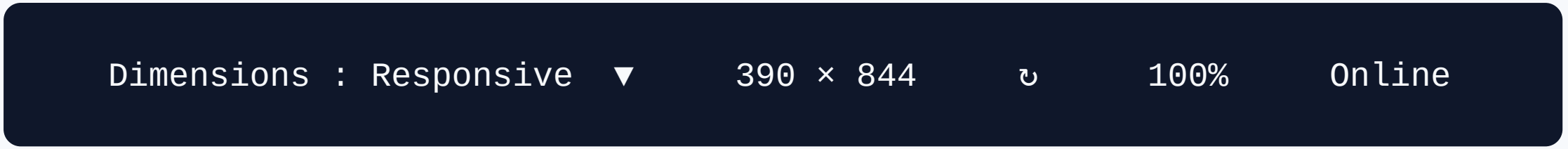
600 px	900 px	1200 px
NON	OUI	NON

768 px ≤ largeur ≤ 1024 px

Étape 3 — Activer le mode responsive

On simule maintenant téléphone et tablette sans quitter le navigateur.

Chrome / Chromium / Edge : F12 (ou Inspecter) → icône téléphone/tablette • Raccourci : Ctrl + Shift + M



1 Choisir un appareil ou « Responsive »

2 Modifier largeur et hauteur

3 Rotation portrait ↔ paysage

4 Garder le zoom à 100%

Firefox : Ctrl + Shift + M ouvre directement le Responsive Design Mode.

Test B4 — portrait / paysage

Utilisez maintenant le mode responsive.

```
@media (orientation: portrait) {
  #portrait { background-color: green; }
}

@media (orientation: landscape) {
  #landscape { background-color: green; }
}
```

Pendant ce test, ignorez les autres boîtes qui peuvent aussi être vertes.

À FAIRE MAINTENANT

1. Ctrl + Shift + M.
2. Choisissez un téléphone, ou « Responsive ».
3. Placez-le en portrait. Regardez #portrait.
4. Cliquez sur Rotation ↻.
5. Regardez maintenant #landscape.

Observation B4 — une condition peut aussi dépendre de l’orientation

PORTRAIT

PAYSAGE

Deux autres exemples

```
@media screen
→ affichage écran

@media print
→ impression /
  aperçu
```

Pour voir print : Ctrl + P

Avant l’exercice — CSS de base + règles conditionnelles

```css
.tile {
    width: 300px;
    background: white;
}
```



```css
@media (max-width: 600px) {
    .tile {
        width: 50%;
    }
}
```

CSS de base
→ actif normalement

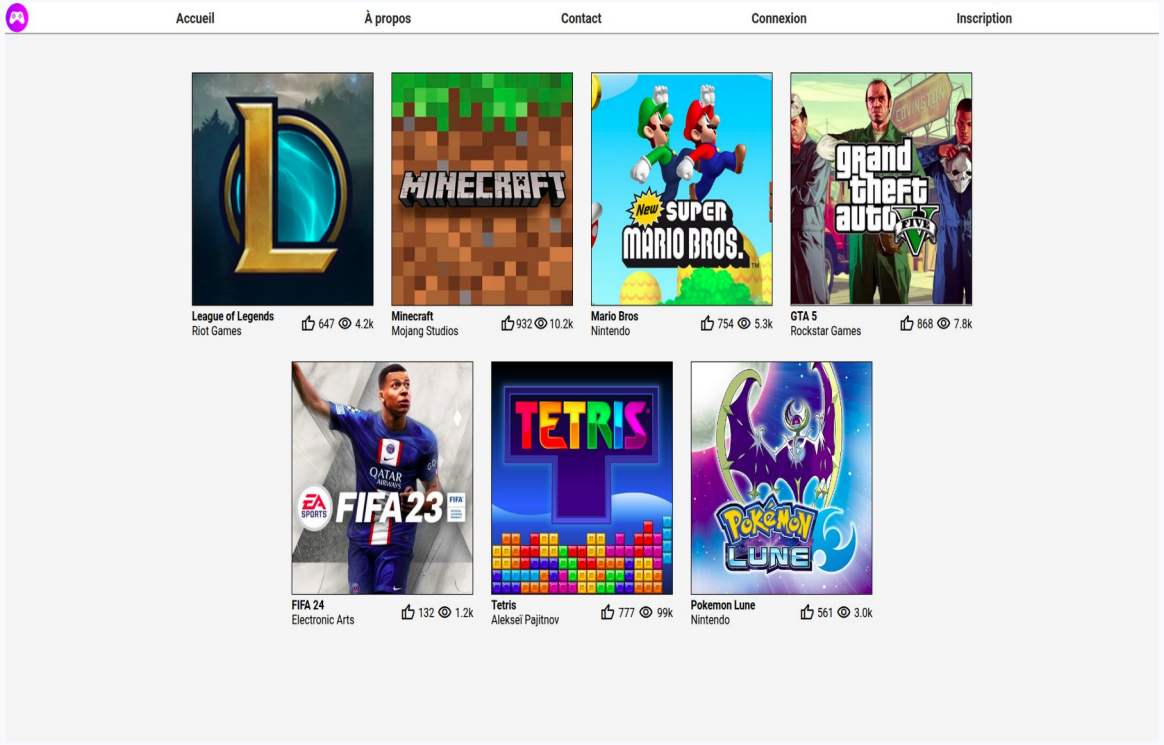
Quand la condition est vraie, width change. background reste white car on ne l’a pas redéfini.

Même HTML. Même sélecteur .tile. On change seulement QUAND certaines propriétés s’appliquent.

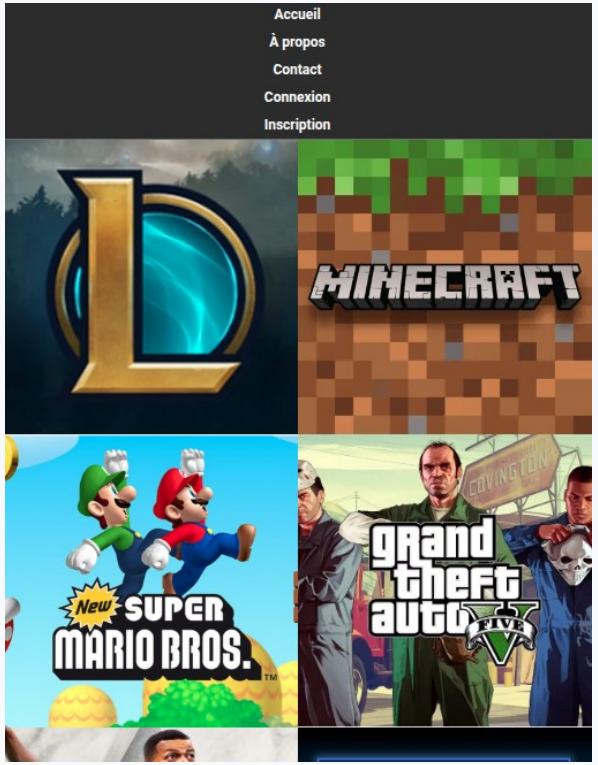
Exercice TD4 — rendre votre page du TD3 responsive

Objectif officiel : la page doit s'adapter quand la fenêtre devient étroite.

ÉCRAN LARGE



ÉCRAN ÉTROIT



Votre travail : obtenir ces deux comportements avec la même page HTML.

Exercice — vos étapes de travail

Cette diapositive peut rester affichée pendant votre travail autonome.

DÉMARRAGE

1. Ouvrez votre page du TD3.

2. Gardez index.html + styles.css ouverts.

3. Commencez par NE PAS changer le HTML.

4. À la fin du CSS, créez votre @media pour petit écran.

5. Testez avec Ctrl + Shift + M.

MISSIONS

1	**HEADER**	vertical + logo/icône masqué
2	**TUILES**	2 colonnes + collées
3	**INFOS**	cachées au repos sur petit écran
4	**HOVER**	afficher les infos sur petit écran
5	**VÉRIFIER**	écran large : hover téléchargement comme avant

Méthode : une mission → tester → seulement ensuite passer à la suivante.

Bilan TD4 — 4 idées suffisent

1

UNITÉS

px fixe ; %, em, vw ont une référence

2

@media

condition : QUAND une règle CSS s'applique

3

SÉLECTEURS

toujours les mêmes : tag, .classe, #id…

4

TEST

redimensionner + mode responsive + portrait/paysage

Le but n'est pas de connaître toutes les propriétés par cœur : savoir lire, cibler, tester et observer.